

Functions

Sunday, 12 July 2026

9:38 AM

- To make code easier to read, the variable type comes after the variable name.

* Multiple Parameters-:

When multiple arguments are of the same type, and are next to each other in the function signature, the type only needs to be declared after last argument.

+ Declaration Syntax-:

Go's declaration are clear, just read them left to right, just like you would in English.

```
x int    p *int    a []int
```

* Passing Variable by value-:

Variable in Go are passed by value.

"pass by value" means that when a variable is passed into function, that function receives a copy of the variable. The function is unable to mutate the caller's original data.

* Ignoring Return values-:

A function can return a value that the caller doesn't care about, we can explicitly ignore variable by using an underscore, or more precisely, the blank identifier _.

Why might you ignore a return value?

Maybe function called `getCircle` returns the centre point and the radius, but you only need the radius for your calculation. In that case, you can ignore the centre point variable.

The Go compiler will return an error if you have any unused variable declarations in your code, so you need to ignore anything you don't intend to use.

+ Named Return values-:

Return values may be given names, and if they are treated the same as if they were new variables defined at the top of the function.

- Named return values are best thought of as a way to document the purpose of the returned values.
- A return statement without arguments return values. This is known as a "naked" return. Naked return statement should be used only in short functions. They can harm readability in longer functions.
- Named return parameters are great for documenting function. we know the the function is returning directly from its signature, no need for a comment.

* Explicit Return-:

Even though a function has named return values if we want to.

* Early Returns-:

Go supports the ability to return early from function. This is a powerful feature that can clean up code, especially when used as guard clauses.

- Guard clauses leverage the ability to return early from a function (or continue through a loop) to make nested conditionals one-dimensional. Instead of using if/else chains, we just return early from the function at the end of each conditional block.

+ Functions As Values-:

Go supports first-class and higher-order functions which are just fancy way of saying "functions as values". Functions are just another type -- like ints and strings and bools.

+ Anonymous Functions-:

Anonymous functions are fun to form in that they have no name. They're useful when defining a function that will only be used once or to create quick closure.

+ Defer-:

The `defer` keyword is a fairly unique feature of Go. It allows a function to be executed automatically just before it's enclosing function returns. The deferred call's argument are evaluated immediately, but the function call is not executed until the surrounding function returns.

Deferred functions are typically used to clean up resources that are no longer being used. Often to close database connection, file handlers and the like.

Defer is a great way to make sure that something happen before function exists, even if there are multiple return statement, a common occurrence in Go.

* Multiple Defers-:

The location of a `defer` statement inside a function matters. The deferred call is registered at the point where `defer` is executed, and it will run when the function returns. If you have multiple `defer` statements in a single function, they are executed in last-in, first out order (the last deferred call run first)

+ Block Scope-:

Go is not function-scoped, it's block-scoped. Variable declared inside a block are only accessible within that block (and it's nested blocks).

Blocks are defined by curly braces {}.

- Functions
- Loops
- if statements
- switch statements
- select statements
- Explicit blocks

* Closures-:

A closure is a function that reference variable from outside its own function body. The function may access and assign to the referenced variables.

* Currying-:

function currying is a concept from functional programming and involves partial application of functions. It allows a function with multiple argument to be transformed into a sequence of functions, each taking a single argument.